

## 1. Introduction

This report presents a comprehensive clustering analysis on the **Iris** and **Wine** datasets from the UCI repository. The goal is to implement, evaluate, and compare multiple clustering algorithms, covering:

- **Partition-based clustering:** K-Means, K-Medoids, Bisecting K-Means
- **Hierarchical clustering:** Agglomerative (Ward linkage), Divisive, Exact Divisive
- **Density-based clustering:** DBSCAN, OPTICS

Evaluation metrics include **external metrics** (Rand Index, Adjusted Rand Index, Mutual Information) and **internal metrics** (Silhouette Coefficient, Calinski-Harabasz Index, Davies-Bouldin Index). Additionally, **cohesion (SSE)** and **separation (SSB)** scores are computed for cluster quality assessment.

## 2. Datasets Used

### 2.1 Iris Dataset

- **Instances:** 150
- **Features:** 4 numeric features (Sepal Length, Sepal Width, Petal Length, Petal Width)
- **Classes:** 3 species of Iris (Setosa, Versicolour, Virginica)

### 2.2 Wine Dataset

- **Instances:** 178
- **Features:** 13 numeric chemical analysis features
- **Classes:** 3 cultivars of wine

```
def load_dataset(name: str):  
    """  
    Load a UCI dataset by name and ensure target labels are integer-encoded  
(0, 1, 2, ...).  
  
    Parameters  
    -----  
    name : str  
        Name of the dataset. Supported values: 'iris', 'wine'.  
  
    Returns  
    -----  
    X : np.ndarray  
        Feature matrix of shape (n_samples, n_features).  
    y : np.ndarray  
        Integer-encoded labels starting from 0.  
    feature_names : list of str  
        Names of the features.  
    target_names : list of str  
        Original target class names.
```

```

Raises
-----
ValueError
    If an unsupported dataset name is provided.
"""
name = name.lower().strip()

if name == 'iris':
    data = datasets.load_iris()
elif name == 'wine':
    data = datasets.load_wine()
else:
    raise ValueError(f"Unsupported dataset name: {name}. Use 'iris' or
'wine'.")

X = data.data
y = data.target

# Ensure labels are 0-indexed integers (sklearn already provides them
this way, but enforce for safety)
unique_labels, y_mapped = np.unique(y, return_inverse=True)
y = y_mapped # Now y contains 0,1,2,...

feature_names = list(data.feature_names)
target_names = list(data.target_names)

print(f"Loaded '{name}' dataset: {X.shape[0]} samples, {X.shape[1]}
features, {len(np.unique(y))} classes.")
return X, y, feature_names, target_names

```

## Preprocessing steps:

- Labels mapped to integers (0,1,2...)
- Feature scaling using **StandardScaler**

```
def preprocess_data(X):
    """
    Standardize features to zero mean and unit variance.

    Parameters
    -----
    X : np.ndarray
        Raw feature matrix of shape (n_samples, n_features).

    Returns
    -----
    X_scaled : np.ndarray
        Scaled feature matrix with mean  $\approx 0$  and std  $\approx 1$ .
    scaler : StandardScaler
        Fitted scaler instance for potential inverse transformation.
    """
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Display summary statistics for verification
    print("Preprocessing complete:")
    print(f"  - Mean of each feature after scaling (approx):
{np.round(X_scaled.mean(axis=0), 4)}")
    print(f"  - Std of each feature after scaling (approx):
{np.round(X_scaled.std(axis=0), 4)}")

    return X_scaled, scaler
```

- Optional PCA visualization for 2D cluster plots

```
def plot_clusters(X, y_pred, title="Clusters"):
    pca = PCA(n_components=2)
    X_2d = pca.fit_transform(X)
    plt.figure(figsize=(6,5))
    plt.scatter(X_2d[:,0], X_2d[:,1], c=y_pred, cmap='tab10', s=50)
    plt.title(title)
    plt.show()
```

## 3. Methods and Code Implementation

### 3.1 Partition-Based Clustering

- **K-Means:** standard iterative centroid update

```
def run_kmeans(X, y_true, k=3, n_init=10, random_state=0):
    """
    Perform K-Means clustering and evaluate performance.

    Parameters
    -----
    X : np.ndarray
        Scaled feature matrix of shape (n_samples, n_features).
    y_true : np.ndarray
        True class labels (for external evaluation).
    k : int, default=3
        Number of clusters.
    n_init : int, default=10
        Number of random initializations.
    random_state : int, default=0
        Random seed for reproducibility.

    Returns
    -----
    results : dict
        Dictionary containing evaluation metrics and K-Means
parameters.
    y_pred : np.ndarray
        Predicted cluster labels.
    centers : np.ndarray
        Cluster centroids.
    """
    # Initialize and fit K-Means
    kmeans = KMeans(
        n_clusters=k,
        init='k-means++',      # uses the K-means++ initialization
method
        n_init=n_init,
        max_iter=300,
        random_state=random_state
    )

    y_pred = kmeans.fit_predict(X)
```

```
centers = kmeans.cluster_centers_  
  
# Evaluate clustering  
eval_results = evaluate_clustering(X, y_true, y_pred)  
  
# Add algorithm info  
eval_results["Algorithm"] = f"KMeans (k={k})"  
  
return eval_results, y_pred, centers
```

- **K-Medoids**: using PyClustering library

```
def run_kmedoids(X, y_true, k=3, metric='euclidean', random_state=0):
    """
    Perform K-Medoids (PAM) clustering using pyclustering implementation.

    Parameters
    -----
    X : np.ndarray
        Scaled feature matrix (n_samples, n_features).
    y_true : np.ndarray
        True class labels (for external evaluation).
    k : int, default=3
        Number of clusters.
    metric : str, default='euclidean'
        Distance metric ('euclidean', 'manhattan', etc.)
    random_state : int, default=0
        Random seed.

    Returns
    -----
    results : dict
        Evaluation metrics.
    y_pred : np.ndarray
        Predicted cluster labels.
    medoids : list
        Indices of medoid samples.
    """
    np.random.seed(random_state)
    n_samples = X.shape[0]

    # Randomly choose initial medoids
    initial_medoids = np.random.choice(n_samples, k,
    replace=False).tolist()

    # Configure distance metric
    if metric == 'euclidean':
        metric_func = distance_metric(type_metric.EUCLIDEAN)
    elif metric == 'manhattan':
        metric_func = distance_metric(type_metric.MANHATTAN)
    else:
        raise ValueError("Unsupported metric. Use 'euclidean' or
    'manhattan'.")
```

```
# Run PAM (K-Medoids)
kmedoids_instance = kmedoids(X, initial_medoids, metric=metric_func)
kmedoids_instance.process()

clusters = kmedoids_instance.get_clusters()
medoids = kmedoids_instance.get_medoids()

# Convert cluster list to label array
y_pred = np.zeros(n_samples, dtype=int)
for cluster_idx, cluster_points in enumerate(clusters):
    y_pred[cluster_points] = cluster_idx

# Evaluate
eval_results = evaluate_clustering(X, y_true, y_pred)
eval_results["Algorithm"] = f"KMedoids (k={k})"

return eval_results, y_pred, medoids
```



- **Bisecting K-Means:** recursive binary splits

```
def run_bisecting_kmeans(X, y_true, k=3, n_init=10, random_state=0,
max_iter=10):
    """
    Perform Bisecting K-Means clustering and evaluate performance.

    Parameters
    -----
    X : np.ndarray
        Scaled feature matrix (n_samples, n_features).
    y_true : np.ndarray
        True class labels (for evaluation).
    k : int, default=3
        Desired number of final clusters.
    n_init : int, default=10
        Number of initializations for K-Means at each split.
    random_state : int, default=0
        Random seed for reproducibility.
    max_iter : int, default=10
        Maximum number of bisection iterations per cluster.

    Returns
    -----
    results : dict
        Evaluation metrics for Bisecting K-Means.
    y_pred : np.ndarray
        Predicted cluster labels.
    cluster_centers : list
        List of cluster centroids.
    """

    np.random.seed(random_state)
    n_samples = X.shape[0]

    # Start with one cluster containing all points
    clusters = {0: np.arange(n_samples)}

    # Keep splitting until desired number of clusters reached
    while len(clusters) < k:
        # Step 1: Select the cluster with largest SSE
        sse_per_cluster = {}
        for cid, indices in clusters.items():
            subset = X[indices]
```

```

        center = np.mean(subset, axis=0)
        sse = np.sum((subset - center) ** 2)
        sse_per_cluster[cid] = sse

# Select cluster with largest SSE
cluster_to_split = max(sse_per_cluster, key=sse_per_cluster.get)
indices_to_split = clusters[cluster_to_split]
X_subset = X[indices_to_split]

# Step 2: Bisect (apply K-Means with k=2)
best_labels = None
best_inertia = np.inf

for i in range(max_iter):
    kmeans_2 = KMeans(
        n_clusters=2,
        n_init=n_init,
        random_state=random_state + i
    ).fit(X_subset)

    if kmeans_2.inertia_ < best_inertia:
        best_labels = kmeans_2.labels_
        best_inertia = kmeans_2.inertia_

# Step 3: Replace old cluster with new two clusters
new_cluster_id = max(clusters.keys()) + 1
clusters[cluster_to_split] = indices_to_split[best_labels == 0]
clusters[new_cluster_id] = indices_to_split[best_labels == 1]

# Construct final label array
y_pred = np.zeros(n_samples, dtype=int)
for cid, indices in clusters.items():
    y_pred[indices] = cid

# Compute cluster centers
cluster_centers = [np.mean(X[indices], axis=0) for indices in
clusters.values()]

# Evaluate
eval_results = evaluate_clustering(X, y_true, y_pred)
eval_results["Algorithm"] = f"Bisecting KMeans (k={k})"

return eval_results, y_pred, cluster_centers

```

## 3.2 Hierarchical Clustering

- **Agglomerative (Ward)**: bottom-up merging based on SSE

```
def run_hierarchical(X, y_true, k=3, linkage_method='ward',
plot_dendrogram=True):
    """
    Perform Agglomerative Hierarchical Clustering and evaluate performance.
    Optionally plot the dendrogram.

    Parameters
    -----
    X : np.ndarray
        Scaled feature matrix (n_samples, n_features).
    y_true : np.ndarray
        True class labels (for evaluation).
    k : int, default=3
        Desired number of clusters.
    linkage_method : str, default='ward'
        Linkage criterion ('ward', 'complete', 'average', 'single').
    plot_dendrogram : bool, default=True
        Whether to display the dendrogram plot.

    Returns
    -----
    results : dict
        Evaluation metrics and algorithm info.
    y_pred : np.ndarray
        Predicted cluster labels.
    """
    # Perform Agglomerative Clustering
    hc = AgglomerativeClustering(
        n_clusters=k,
        metric='euclidean',
        linkage=linkage_method
    )
    y_pred = hc.fit_predict(X)

    # Evaluate clustering
    eval_results = evaluate_clustering(X, y_true, y_pred)
    eval_results["Algorithm"] = f"Hierarchical ({linkage_method}, k={k})"

    # Optionally plot dendrogram
    if plot_dendrogram:
        plt.figure(figsize=(10, 5))
```

```
Z = linkage(X, method=linkage_method)
dendrogram(Z, truncate_mode='level', p=5)
plt.title(f"Dendrogram ({linkage_method} linkage)")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.show()
```

```
return eval_results, y_pred
```

- **Divisive:** top-down recursive splits

```
def run_divisive_hierarchical(X, y_true, k=3, n_init=10, random_state=0):
    """
    Perform divisive hierarchical clustering using recursive bisecting
    K-Means.

    Parameters
    -----
    X : np.ndarray
        Scaled feature matrix (n_samples, n_features)
    y_true : np.ndarray
        True class labels (for evaluation)
    k : int
        Desired number of clusters
    n_init : int
        Number of initializations for K-Means at each split
    random_state : int
        Random seed

    Returns
    -----
    eval_results : dict
        Clustering evaluation metrics
    y_pred : np.ndarray
        Predicted cluster labels
    """
    np.random.seed(random_state)
    n_samples = X.shape[0]

    # Each element is a tuple: (indices of cluster points)
    clusters = [np.arange(n_samples)]

    while len(clusters) < k:
        # Select cluster with largest SSE to split
        sse_per_cluster = []
        for idx, indices in enumerate(clusters):
            subset = X[indices]
            center = np.mean(subset, axis=0)
            sse = np.sum((subset - center) ** 2)
            sse_per_cluster.append(sse)
        cluster_to_split_idx = np.argmax(sse_per_cluster)
        indices_to_split = clusters.pop(cluster_to_split_idx)
        X_subset = X[indices_to_split]
```

```

    # Split into 2 using K-Means
    kmeans_2 = KMeans(n_clusters=2, n_init=n_init,
random_state=random_state)
    labels_split = kmeans_2.fit_predict(X_subset)

    # Append the two new clusters
    clusters.append(indices_to_split[labels_split == 0])
    clusters.append(indices_to_split[labels_split == 1])

# Build final label array
y_pred = np.zeros(n_samples, dtype=int)
for cid, indices in enumerate(clusters):
    y_pred[indices] = cid

# Evaluate clustering
eval_results = evaluate_clustering(X, y_true, y_pred)
eval_results["Algorithm"] = f"Divisive Hierarchical (k={k})"

return eval_results, y_pred

```

### 3.3 Density-Based Clustering

- **DBSCAN**: density-based clustering with eps/min\_samples tuning
- **OPTICS**: density-based with reachability and xi parameters

```
def run_density_based(X, y_true, method='dbscan', eps=0.5, min_samples=5,
xi=0.05, min_cluster_size=5):
    """
    Run density-based clustering (DBSCAN or OPTICS) and evaluate
    performance.

    Parameters
    -----
    X : np.ndarray
        Scaled feature matrix
    y_true : np.ndarray
        True class labels
    method : str, default='dbscan'
        'dbscan' or 'optics'
    eps : float
        Neighborhood radius (DBSCAN)
    min_samples : int
        Minimum samples for core point (DBSCAN)
    xi : float
        Sensitivity parameter for OPTICS (optional)
    min_cluster_size : int
        Minimum cluster size for OPTICS

    Returns
    -----
    eval_results : dict
        Evaluation metrics
    y_pred : np.ndarray
        Cluster labels
    """

    if method.lower() == 'dbscan':
        clusterer = DBSCAN(eps=eps, min_samples=min_samples)
    elif method.lower() == 'optics':
        clusterer = OPTICS(xi=xi, min_samples=min_samples,
min_cluster_size=min_cluster_size)
    else:
        raise ValueError("method must be 'dbscan' or 'optics'")
```

```
y_pred = clusterer.fit_predict(X)

# Optional: relabel noise (-1) as a separate cluster for evaluation
y_pred_eval = np.copy(y_pred)
if -1 in y_pred_eval:
    y_pred_eval[y_pred_eval == -1] = np.max(y_pred_eval) + 1

eval_results = evaluate_clustering(X, y_true, y_pred_eval)
eval_results["Algorithm"] = f"{method.upper()}"

return eval_results, y_pred
```



## 4. Results

### 4.1 Iris Dataset

```
ds_name = 'iris'
all_results = {}

# Load and preprocess
X, y, feat_names, target_names = load_dataset(ds_name)
# print(f"Unique labels ({ds_name}):", np.unique(y))
# print("First 5 samples:\n", X[:5])

X_scaled, _ = preprocess_data(X)

# Store dataset-specific results
results_list = []

# --- Partition-based ---
results_kmeans, y_pred_kmeans, _ = run_kmeans(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_kmeans)

results_kmedoids, y_pred_kmedoids, _ = run_kmedoids(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_kmedoids)

results_bkm, y_pred_bkm, _ = run_bisecting_kmeans(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_bkm)

# --- Hierarchical ---
results_hier, y_pred_hier = run_hierarchical(X_scaled, y,
k=len(np.unique(y)), linkage_method='ward', plot_dendrogram=False)
results_list.append(results_hier)

results_divisive, y_pred_divisive = run_divisive_hierarchical(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_divisive)

results_exact_div, y_pred_exact_div = exact_divisive(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_exact_div)
```

```

# --- Density-based ---
results_dbscan, y_pred_dbscan = run_density_based(X_scaled, y,
method='dbscan', eps=0.45, min_samples=5)
results_list.append(results_dbscan)

results_optics, y_pred_optics = run_density_based(X_scaled, y,
method='optics', xi=0.12, min_samples=5)
results_list.append(results_optics)

# Merge into a single DataFrame
df_results = pd.DataFrame(results_list).set_index("Algorithm")
all_results[ds_name] = df_results

# Display the table
print(f"\n=== {ds_name.upper()} Dataset: Clustering Performance ===\n")
display(df_results)

```

## Iris Results:

Loaded 'iris' dataset: 150 samples, 4 features, 3 classes.

Preprocessing complete:

- Mean of each feature after scaling (approx): [-0. -0. -0. -0.]
- Std of each feature after scaling (approx): [1. 1. 1. 1.]

=== IRIS Dataset: Clustering Performance ===

|                          | Rand     | Adj Rand | MI       | Adj MI   | Norm MI  | Silhouette | Calinski   | Davies   | SSE        | SSB        | Accuracy |
|--------------------------|----------|----------|----------|----------|----------|------------|------------|----------|------------|------------|----------|
| Algorithm                |          |          |          |          |          |            |            |          |            |            |          |
| KMeans (k=3)             | 0.832215 | 0.620135 | 0.724124 | 0.655223 | 0.659487 | 0.459948   | 241.904402 | 0.833595 | 139.820496 | 460.179504 | 0.833333 |
| KMedoids (k=3)           | 0.856823 | 0.676454 | 0.767794 | 0.696651 | 0.700410 | 0.450791   | 238.547924 | 0.831681 | 141.324446 | 458.675554 | 0.866667 |
| Bisecting KMeans (k=3)   | 0.832215 | 0.620135 | 0.724124 | 0.655223 | 0.659487 | 0.459948   | 241.904402 | 0.833595 | 139.820496 | 460.179504 | 0.833333 |
| Hierarchical (ward, k=3) | 0.825235 | 0.615323 | 0.722757 | 0.671286 | 0.675470 | 0.446689   | 222.719164 | 0.803467 | 148.876256 | 451.123744 | 0.826667 |

|                                          |          |          |          |          |          |          |            |          |            |            |          |
|------------------------------------------|----------|----------|----------|----------|----------|----------|------------|----------|------------|------------|----------|
| <b>Divisive Hierarchical (k=3)</b>       | 0.827830 | 0.610073 | 0.716734 | 0.648209 | 0.652558 | 0.459378 | 241.893262 | 0.834000 | 139.825435 | 460.174565 | 0.826667 |
| <b>Exact Divisive Hierarchical (k=3)</b> | 0.827830 | 0.610073 | 0.716734 | 0.648209 | 0.652558 | 0.459378 | 241.893262 | 0.834000 | 139.825435 | 460.174565 | 0.826667 |
| <b>DBSCAN</b>                            | 0.794810 | 0.517100 | 0.709498 | 0.583187 | 0.590347 | 0.180598 | 51.240777  | 3.544692 | 292.270514 | 307.729486 | 0.746667 |
| <b>OPTICS</b>                            | 0.776286 | 0.568116 | 0.636514 | 0.731585 | 0.733680 | 0.581750 | 251.349339 | 0.593313 | 222.361705 | 377.638295 | 0.666667 |

## 4.2 Wine Dataset

```
ds_name = 'wine'

all_results = {}

X, y, feat_names, target_names = load_dataset(ds_name)
# print(f"Unique labels ({ds_name}):", np.unique(y))
# print("First 5 samples:\n", X[:5])

X_scaled, _ = preprocess_data(X)

# Store dataset-specific results
results_list = []

# --- Partition-based ---
results_kmeans, y_pred_kmeans, _ = run_kmeans(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_kmeans)

results_kmedoids, y_pred_kmedoids, _ = run_kmedoids(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_kmedoids)

results_bkm, y_pred_bkm, _ = run_bisecting_kmeans(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_bkm)

# --- Hierarchical ---
results_hier, y_pred_hier = run_hierarchical(X_scaled, y,
k=len(np.unique(y)), linkage_method='ward', plot_dendrogram=False)
results_list.append(results_hier)

results_divisive, y_pred_divisive = run_divisive_hierarchical(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_divisive)

results_exact_div, y_pred_exact_div = exact_divisive(X_scaled, y,
k=len(np.unique(y)))
results_list.append(results_exact_div)

# --- Density-based ---
results_dbscan, y_pred_dbscan = run_density_based(X_scaled, y,
method='dbscan', eps=2.25, min_samples=5)
```

```

results_list.append(results_dbscan)

results_optics, y_pred_optics = run_density_based(X_scaled, y,
method='optics', xi=0.05, min_samples=5)
results_list.append(results_optics)

# Merge into a single DataFrame
df_results = pd.DataFrame(results_list).set_index("Algorithm")
all_results[ds_name] = df_results

# Display the table
print(f"\n=== {ds_name.upper()} Dataset: Clustering Performance ===\n")
display(df_results)

```

## Wine Results:

Loaded 'wine' dataset: 178 samples, 13 features, 3 classes.  
Preprocessing complete:  
- Mean of each feature after scaling (approx): [ 0. 0. -0. -0. -0. -0. 0. -0.  
-0. -0. 0. 0. -0.]  
- Std of each feature after scaling (approx): [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]

=== WINE Dataset: Clustering Performance ===

|                                              | Rand         | Adj<br>Rand  | MI           | Adj<br>MI    | Norm<br>MI   | Silho<br>uette | Calin<br>ski  | Davi<br>es   | SSE             | SSB             | Accur<br>acy |
|----------------------------------------------|--------------|--------------|--------------|--------------|--------------|----------------|---------------|--------------|-----------------|-----------------|--------------|
| Algorit<br>hm                                |              |              |              |              |              |                |               |              |                 |                 |              |
| <b>KMeans<br/>(k=3)</b>                      | 0.95<br>4294 | 0.89<br>7495 | 0.95<br>4458 | 0.87<br>4579 | 0.87<br>5894 | 0.284<br>859   | 70.94<br>0008 | 1.38<br>9188 | 1277.9<br>28489 | 1036.0<br>71511 | 0.966<br>292 |
| <b>KMedoid<br/>s (k=3)</b>                   | 0.87<br>7484 | 0.72<br>6341 | 0.82<br>1479 | 0.75<br>3986 | 0.75<br>6574 | 0.265<br>977   | 66.75<br>1966 | 1.41<br>5990 | 1312.6<br>25089 | 1001.3<br>74911 | 0.904<br>494 |
| <b>Bisecti<br/>ng<br/>KMeans<br/>(k=3)</b>   | 0.81<br>2988 | 0.59<br>0631 | 0.75<br>1549 | 0.70<br>1892 | 0.70<br>5090 | 0.234<br>076   | 59.74<br>2457 | 1.52<br>0319 | 1375.1<br>12890 | 938.88<br>7110  | 0.837<br>079 |
| <b>Hierarc<br/>hical<br/>(ward,<br/>k=3)</b> | 0.90<br>6494 | 0.78<br>9933 | 0.85<br>8437 | 0.78<br>4208 | 0.78<br>6465 | 0.277<br>444   | 67.64<br>7468 | 1.41<br>8592 | 1305.0<br>48695 | 1008.9<br>51305 | 0.926<br>966 |

|                                          |          |          |          |          |          |           |           |          |             |            |          |
|------------------------------------------|----------|----------|----------|----------|----------|-----------|-----------|----------|-------------|------------|----------|
| <b>Divisive Hierarchical (k=3)</b>       | 0.868977 | 0.706152 | 0.808709 | 0.739411 | 0.742142 | 0.267797  | 66.394571 | 1.428594 | 1315.673446 | 998.326554 | 0.887640 |
| <b>Exact Divisive Hierarchical (k=3)</b> | 0.868977 | 0.706152 | 0.808709 | 0.739411 | 0.742142 | 0.267797  | 66.394571 | 1.428594 | 1315.673446 | 998.326554 | 0.887640 |
| <b>DBSCAN</b>                            | 0.722656 | 0.396239 | 0.512780 | 0.477828 | 0.483461 | 0.165469  | 28.633243 | 3.627038 | 1743.471512 | 570.528488 | 0.713483 |
| <b>OPTICS</b>                            | 0.439154 | 0.035817 | 0.162126 | 0.167991 | 0.194946 | -0.133640 | 5.057149  | 1.619371 | 2071.753404 | 242.246596 | 0.410112 |

## 5. Discussion

- **Partition-based vs. Hierarchical:**

- K-Means, K-Medoids, and Bisecting K-Means generally give similar high performance.
- Agglomerative and divisive hierarchical clustering provide additional interpretability via dendrograms.

- **Density-based methods:**

- DBSCAN shows slight improvements for Iris after careful parameter tuning.
- OPTICS underperforming on small uniform datasets.
- On Wine, density-based clustering struggles with high-dimensional data.

- **Metric Analysis:**

- **External metrics** (Rand, Adjusted Rand, Mutual Info) confirm alignment with true labels.
- **Internal metrics** (Silhouette, Calinski-Harabasz, Davies-Bouldin) measure cluster cohesion and separation.
- **SSE and SSB** help quantify intra-cluster and inter-cluster variance.

- **Limitations:**

- Density-based clustering is sensitive to hyperparameters.
- Divisive hierarchical clustering is computationally expensive for large datasets.

## 6. Summary & Conclusion

- All three clustering categories were implemented and evaluated on **Iris** and **Wine** datasets.
- **Partition-based and hierarchical algorithms** generally outperform density-based methods on these datasets.
- **Density based clustering could not** achieve the assignment goal of accuracy  $\geq 0.8$  even with eps tuning.